



PROJE TASARIM RAPORU

Ders: Bil 204 / Yazılım Mühendisliği

Bölüm/Şube: Bilgisayar Mühendisliği

Grup Adı: Film Arşivleme Puanlama

Proje Adı: CineRate

Teslim Tarihi: 04.05.2026

Grup Üyeleri:

- Abdulkadir Kırmızı (24100011018)
- Arda Burak Yiğit (24100011019)
- Burak Turan (24100011050)
- Halil Üstündağ (24100011071)
- Mustafa Alper Ataş (24100011073)
- Ömer Şahin (24100011068)

İçindekiler

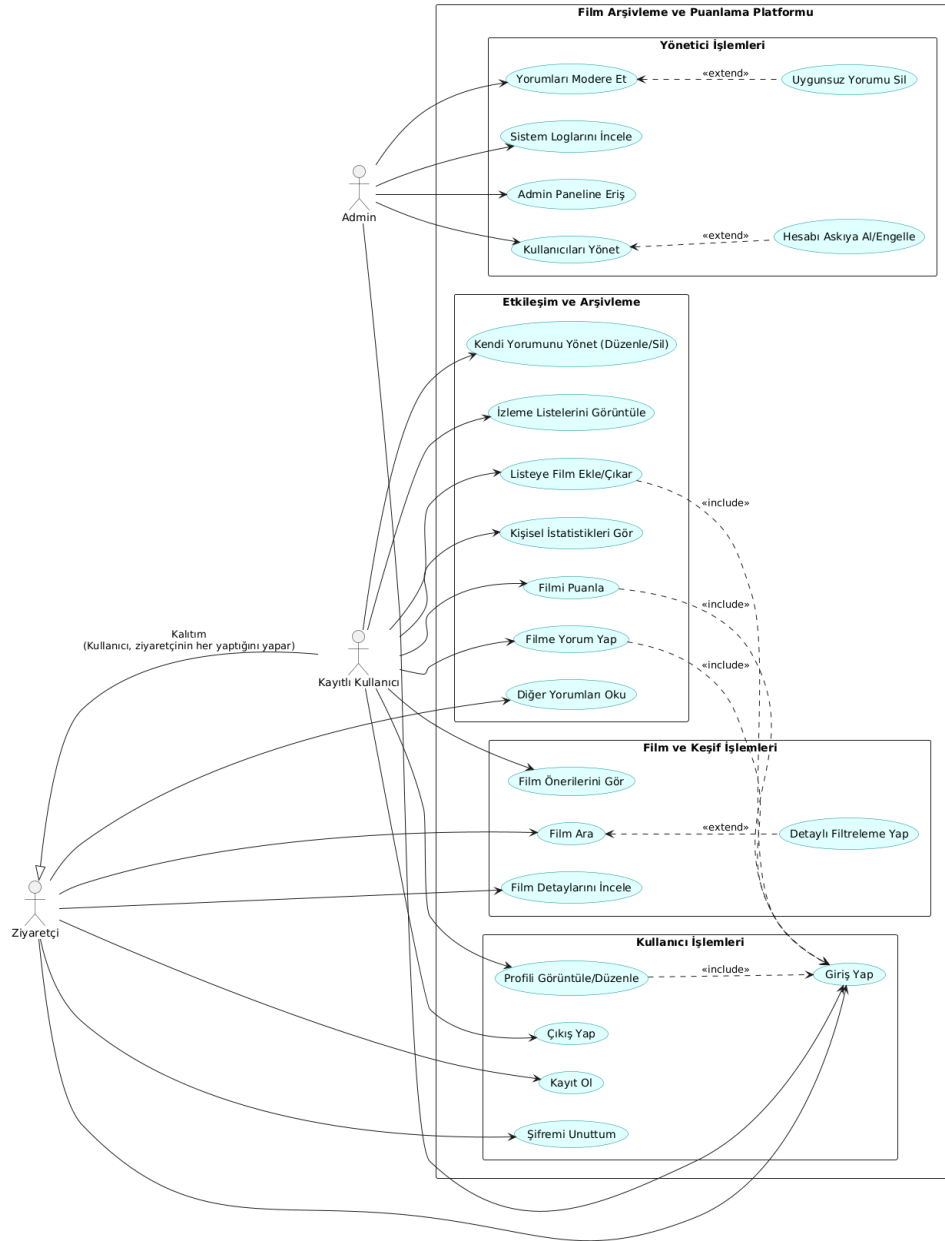
1. Giriş.....	2
1.1. Sistemin Amacı	3
1.2. Tasarım Hedefleri.....	4
1.2.1. Kullanılabilirlik ve Arayüz.....	4
1.2.2. Performans	4
1.2.3. Güvenlik.....	4
1.2.4. Modülerlik ve Bakım Edilebilirlik	4
2. Üst Düzey Yazılım Mimarisi	5
2.1. Alt Sistem Ayırıştırması.....	5
2.2. Donanım/Yazılım Eşlemesi.....	6
2.2.1. Donanım Bileşenleri ve Düşümler.....	7
2.2.2. Yazılım Bileşenlerinin Yerleşimi	8
2.3. Kalıcı Veri Yönetimi.....	8
2.3.1. Veritabanı Seçimi ve Depolama Stratejisi	8
2.3.2. Varlık-İlişki (ER) Yapısı.....	9
2.3.3. Veri Saklama ve API Entegrasyonu Ayrımı	10
2.4. Erişim Kontrolü ve Güvenlik.....	10
2.4.1. Şifreleme ve Kimlik Doğrulama	10
2.4.2. Rol Tabanlı Yetkilendirme.....	10
2.4.3. Veritabanı ve Ağ Güvenliği	11
2.5. Küresel Kontrol Akışı	11
2.6. Sınır Koşulları.....	13
2.6.1. Başlatma.....	13
2.6.2. Sonlandırma	13
2.6.3. Hata Durumları	14
3. Alt Düzey Tasarım	14
3.1. Nesne Tasarım Kararları.....	14
3.2. Son Nesne Tasarımı	15
3.3. Paketler / Katmanlar.....	15
3.3.1. Dış Paketler	16
3.3.2. İç Paketler.....	16
3.4. Sınıf Arayüzleri.....	16
3.4.1. User (Kullanıcı) Sınıfı Metotları	17
3.4.2. Movie (Film) ve API Servis Metotları	17
3.4.3. Interaction (Etkileşim) Metotları.....	18
3.5. Tasarım Desenleri.....	18
3.5.1. Model-View-Template (MVT) Deseni.....	18
3.5.2. Facade (Cephe) Deseni.....	19
3.5.3. Singleton (Tek Nesne) Deseni.....	19
3.5.4. Proxy (Vekil) Deseni.....	19
Sözlük (Glossary)	20
Referanslar	21

1. Giriş

Bu bölümde, web tabanlı film puanlama, arşivleme ve yorumlama platformu projesinin temel amacı ve hedeflenen tasarım kriterleri sunulmaktadır.

1.1. Sistemin Amacı

Projenin temel amacı, sinema izleyicilerinin izledikleri veya izlemek istedikleri yapımları manuel yöntemlerle takip etme zorluğunu ortadan kaldırmaktır. Sistem, tüm verilerin güvenli ve ilişkisel bir veri tabanında saklandığı koordineli bir ekosistem sunar. Platform; harici veri kaynaklarıyla (Örn: TMDB API) entegre çalışarak kullanıcılara güncel film bilgilerini sunmayı, kişisel film kütüphanelerini oluşturmalarına olanak tanımayı ve yorum/puan aracılığıyla topluluk etkileşimini sağlamayı hedefler.



1.2. Tasarım Hedefleri

Projenin başarılı olabilmesi için belirlenen temel işlevsel olmayan gereksinimler ve tasarım hedefleri aşağıda listelenmiştir:

1.2.1. Kullanılabilirlik ve Arayüz (Usability):

Platform, her seviyeden kullanıcının rahatça gezinebileceği sezgisel bir arayüze sahip olmalıdır. Dinamik HTML şablonları (Jinja2) kullanılarak, film arama ve listeleme ekranları modern bir tasarımla sunulacak; kullanıcı deneyimi ön planda tutulacaktır.

1.2.2. Performans (Performance):

Sistem, TMDB API üzerinden veri çekerken gereksiz ağ trafiği yaratmamak adına yalnızca ihtiyaç duyulan JSON verilerini (afiş, yıl, başlık) parse edecek şekilde tasarlanmıştır. Veritabanı işlemleri, hafif ve hızlı okuma/yazma kapasitesine sahip SQLite üzerinde SQLAlchemy ORM ile optimize edilerek gerçekleştirilecektir.

1.2.3. Güvenlik (Security):

Kullanıcı verilerinin güvenliği en yüksek önceliklerden biridir. Kullanıcı şifreleri veritabanına düz metin (plain text) olarak kaydedilmeyecek, Werkzeug kütüphanesi kullanılarak kriptografik özetleme (hashing) işleminden geçirilecektir. Ayrıca SQLAlchemy kullanımı sayesinde SQL Enjeksiyonu (SQL Injection) saldırılarına karşı doğal bir koruma sağlanacaktır.

1.2.4. Modülerlik ve Bakım Edilebilirlik (Modularity & Maintainability):

Proje, Flask framework'ünün sunduğu MVT (Model-View-Template) mimarisine uygun olarak modüler bir yapıda tasarlanmıştır. Veritabanı tabloları (models.py), sayfa rotaları/iş mantığı (views.py) ve harici API istekleri birbirinden tamamen izole edilerek kodun gelecekteki bakımı ve geliştirilebilirliği kolaylaştırılmıştır.

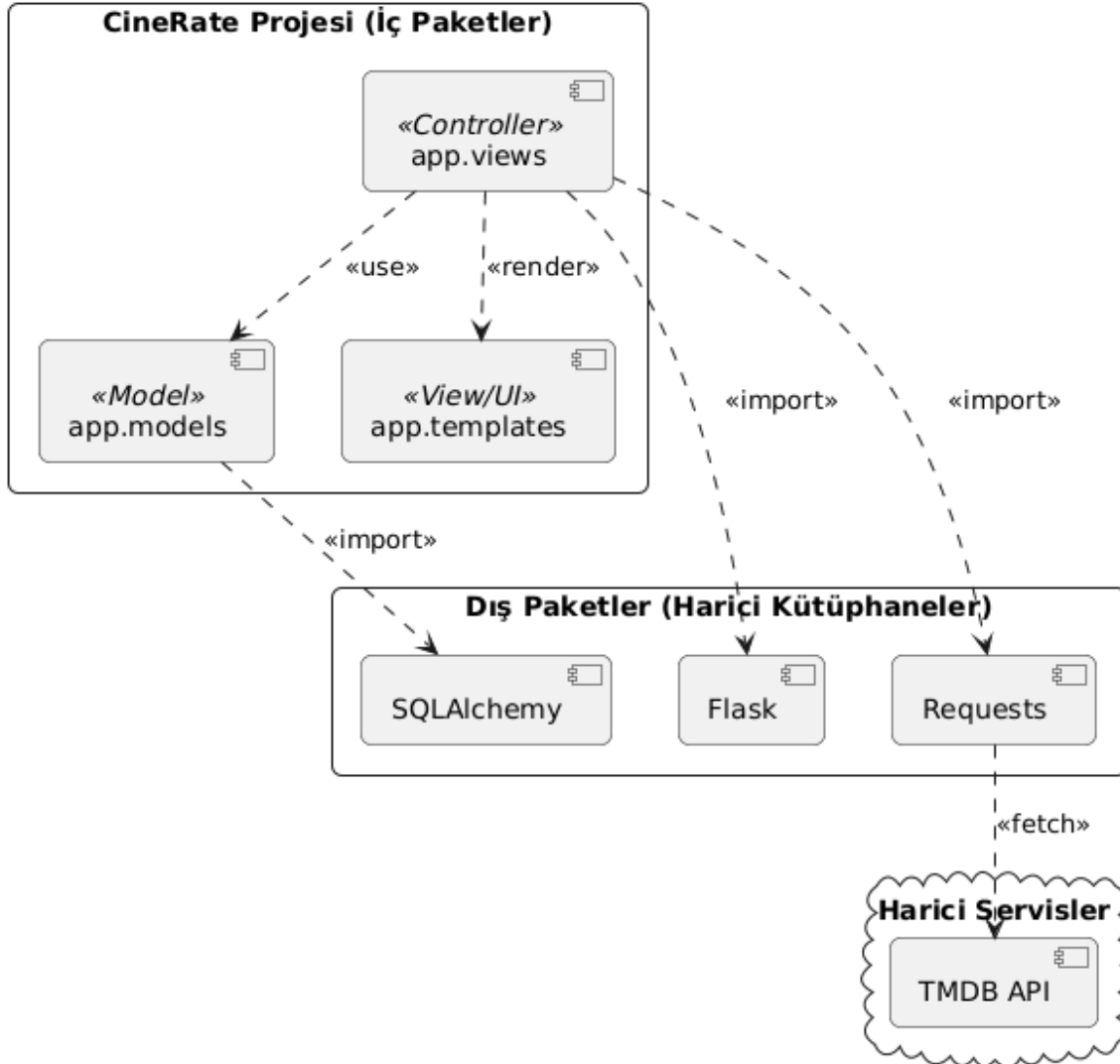
2. Üst Düzey Yazılım Mimarisi

Bu bölümde sistemin genel mimari yapısı, bileşenlerin birbirleriyle olan ilişkisi ve veri akış süreçleri detaylandırılmaktadır.

2.1. Alt Sistem Ayırıştırması(Subsystem Decomposition):

CineRate sistemi, projenin karmaşıklığını yönetmek, kodun yeniden kullanılabilirliğini artırmak ve sürdürülebilir bir geliştirme ortamı sağlamak amacıyla modüler bir yapıda tasarlanmıştır. Sistem, klasik MVC (Model-View-Controller) mimarisinin Flask framework'üne uyarlanmış hali olan **MVT (Model-View-Template)** tasarım desenini temel almaktadır.

Yazılım mimarimiz; kodun iş mantığını, veri yönetimini ve kullanıcı arayüzünü birbirinden tamamen izole eden alt paketlere bölünmüştür. Aşağıdaki paket diyagramında, sistemin iç modülleri, harici kütüphane bağımlılıkları ve dış servisler arasındaki etkileşim gösterilmektedir.



CineRate Projesi (İç Paketler)

Sistemin çekirdek iş mantığını barındıran, kendi geliştirdiğimiz yerel modüllerdir:

- `app.views` (Controller Katmanı): Sistemin beynidir. İstemciden (tarayıcıdan) gelen HTTP isteklerini yakalar, TMDB API'den veri çeker, veritabanı işlemlerini tetikler ve elde ettiği sonuçları HTML şablonlarına göndererek sayfa yönlendirmelerini (<<render>>) sağlar.
- `app.models` (Model Katmanı): Veritabanı tablolarının (User, Movie, Review) nesne yönelimli programlama (OOP) mantığıyla Python sınıfları olarak tanımlandığı pakettir.
- `app.templates` (View/UI Katmanı): Kullanıcının etkileşime girdiği görsel arayüzü temsil eder. Flask'ın Jinja2 motoru ile güçlendirilmiş dinamik HTML dosyalarını barındırır.

Dış Paketler (Harici Kütüphaneler)

Projenin altyapısını güçlendirmek için entegre edilen üçüncü parti Python kütüphaneleridir:

- `Flask`: Web sunucusunun ayağa kaldırılması ve HTTP yönlendirmelerinin yapılması için kullanılan ana çatıdır.
- `SQLAlchemy`: Veritabanı ile uygulama arasındaki köprüyü kuran ORM (Object Relational Mapping) aracıdır. Karmaşık SQL sorguları yazmak yerine, verilerin Python nesneleri aracılığıyla (`app.models` üzerinden) yönetilmesini sağlar.
- `Requests`: Sistemin dış dünyayla iletişim kurmasını sağlayan HTTP istemci kütüphanesidir.

Harici Servisler (Bulut Bileşenleri)

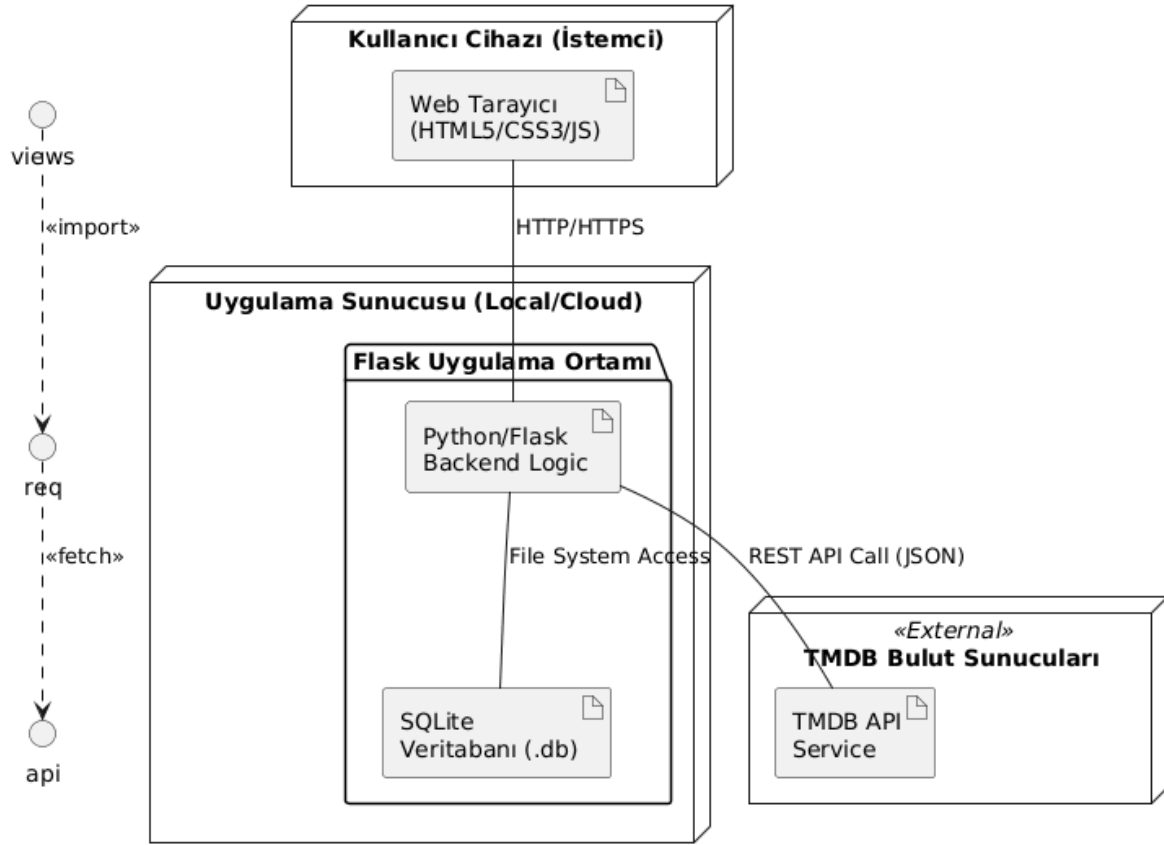
- `TMDB API`: Projenin statik bir veritabanına hapsolmasını engelleyen dış veri sağlayıcısıdır. `app.views` üzerinden `Requests` modülü kullanılarak bu bulut servisine anlık <<fetch>> (getirme) istekleri atılır ve güncel sinema verileri JSON formatında sisteme entegre edilir.

Bu ayrıştırma sayesinde sistemdeki herhangi bir arayüz değişikliği veritabanı katmanını, API katmanındaki bir güncelleme ise kullanıcı arayüzünü etkilemeden (gevşek bağlılık - loose coupling prensibiyle) güvenle yönetilebilmektedir.

2.2. Donanım/Yazılım Eşlemesi (Hardware/Software Mapping)

Bu bölümde, CineRate sistemini oluşturan yazılım bileşenlerinin hangi donanım düğümleri (nodes) üzerinde konumlandırıldığı ve bu düğümler arasındaki fiziksel/mantıksal bağlantılar detaylandırılmaktadır. Sistem, istemci-sunucu (client-server) modelini temel alan dağıtık bir yapı sergilemektedir.

Aşağıdaki Dağıtım Diyagramı (Deployment Diagram), sistemin çalışma zamanındaki fiziksel mimarisini ve bileşenlerin yerleşimini göstermektedir.



2.2.1. Donanım Bileşenleri ve Düğümler

1. **İstemci Cihazı (Client PC/Mobile):** Kullanıcının platforma erişmek için kullandığı donanımdır. Herhangi bir işletim sistemine ve modern bir web tarayıcısına sahip cihazlar bu kapsamdadır.
2. **Uygulama Sunucusu (Application Server):** Python çalışma ortamının ve Flask uygulamasının barındırıldığı merkezi düğümdür. Geliştirme aşamasında yerel makine (localhost) olan bu düğüm, canlı ortamda bir Bulut Sunucusu (VPS - Virtual Private Server) olarak görev yapar.
3. **Harici API Bulutu (External API Cloud):** TMDb servislerinin çalıştığı, sistemin dışında konumlanan ve HTTPS protokolü üzerinden veri sağlayan düğümdür.

2.2.2. Yazılım Bileşenlerinin Yerleşimi

- **İstemci Tarafı:** Donanım üzerinde çalışan **Web Tarayıcı (Browser)**; sunucudan gelen HTML, CSS ve JavaScript çıktılarını yorumlayarak kullanıcıya sunar.
- **Sunucu Tarafı:**
 - **Python/Flask Runtime:** Tüm iş mantığının ve API entegrasyonunun koordine edildiği ana yazılım katmanıdır.
 - **SQLite Veritabanı Dosyası:** Veri katmanı, harici bir sunucu gerektirmeksizin doğrudan uygulama sunucusu içindeki dosya sisteminde kalıcı olarak saklanır.
- **Bağlantı Protokolleri:** Düğümler arasındaki tüm iletişim, veri güvenliğini sağlamak amacıyla standart **HTTP/HTTPS** protokolleri üzerinden gerçekleştirilir.

2.3. Kalıcı Veri Yönetimi (Persistent Data Management)

CineRate platformunda üretilen verilerin tutarlılığı, güvenliği ve uzun süreli saklanması için ilişkisel bir veritabanı yönetim sistemi (RDBMS) stratejisi benimsenmiştir. Bu bölümde, sistemdeki veri depolama yapısı ve tercih edilen teknolojiler detaylandırılmaktadır.

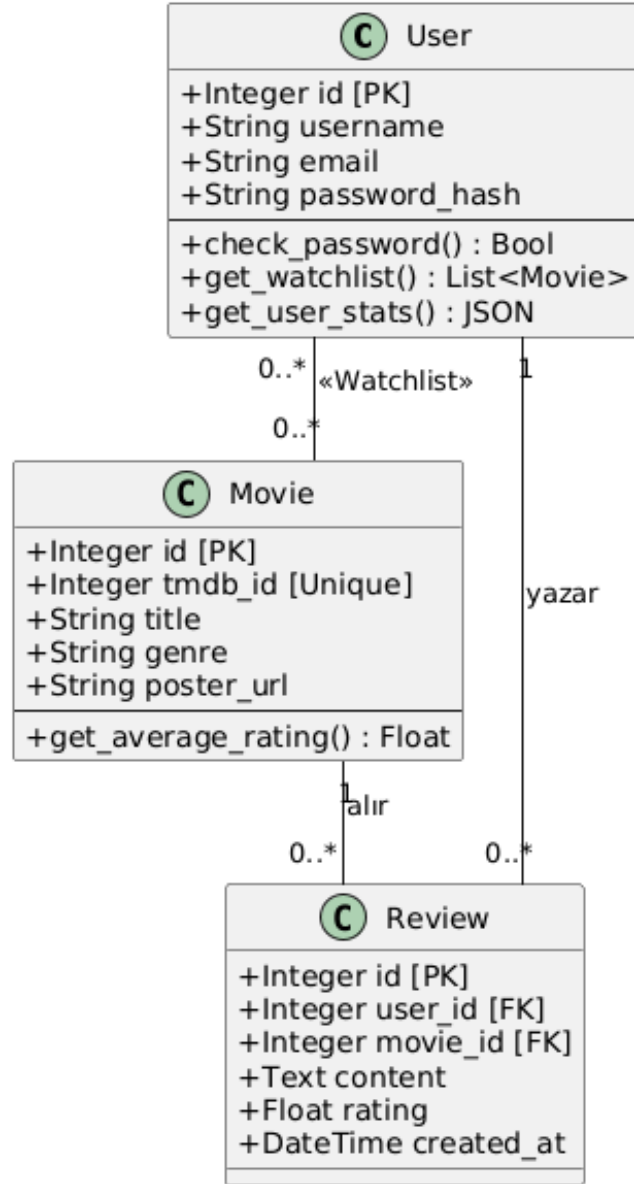
2.3.1. Veritabanı Seçimi ve Depolama Stratejisi

Sistemde veritabanı motoru olarak **SQLite** tercih edilmiştir. Bu seçimin temel nedenleri; konfigürasyon gerektirmeyen "zero-config" yapısı, uygulama dosyalarıyla birlikte taşınabilir olması ve projenin mevcut ölçeği için yüksek performanslı okuma/yazma kapasitesi sunmasıdır.

Veri yönetimi, doğrudan SQL sorguları yazmak yerine **SQLAlchemy ORM** katmanı üzerinden gerçekleştirilir. Bu tercih, veritabanı tablolarının Python sınıfları olarak yönetilmesini sağlayarak "Nesne-İlişkisel Eşleşme" (Object-Relational Mapping) avantajlarından yararlanmamıza ve veri tabanı bağımsız bir yapı kurmamıza olanak tanır.

2.3.2. Varlık-İlişki (ER) Yapısı

Sistemdeki temel veri varlıkları (User, Movie, Review) arasındaki ilişkiler, veritabanı normalizasyon kurallarına uygun olarak tasarlanmıştır.



Diyagramda görülen ilişkisel mantık şu şekildedir:

- **User (Kullanıcı):** Sistemin temel aktörüdür. Bir kullanıcı birden fazla yorum yapabilir ve puan verebilir (Bire-Çok / One-to-Many ilişkisi).
- **Movie (Film):** TMDB API'den gelen verilerin yerel sistemdeki temsilidir. Bir film, farklı kullanıcılar tarafından birçok kez yorumlanabilir (Bire-Çok ilişkisi).
- **Review (Yorum ve Puan):** Kullanıcı ve Film tabloları arasındaki köprüyü oluşturur. Her yorum, mutlaka bir kullanıcıya (user_id) ve bir filme (movie_id) yabancı anahtar (Foreign Key) ile bağlıdır.

2.3.3. Veri Saklama ve API Entegrasyonu Ayrımı

Sistemde gereksiz veri depolamasını önlemek amacıyla "Hibrit Depolama" yöntemi izlenir:

- **Kalıcı Depolananlar:** Kullanıcı bilgileri, şifre özetleri (hashes), yorum metinleri, kullanıcı puanları ve TMDB film kimlik numaraları (tmdb_id).
- **Dinamik Çekilenler:** Film afişleri (poster path), geniş özetler ve oyuncu kadrosu gibi sık güncellenen veya büyük boyutlu medya verileri veritabanında saklanmaz; her oturumda TMDB API'den anlık olarak getirilir.

2.4. Erişim Kontrolü ve Güvenlik (Access Control and Security)

CineRate platformunda, kullanıcı verilerinin gizliliği ve sistemin bütünlüğü kritik önem taşımaktadır. Bu doğrultuda, kimlik doğrulama (authentication) ve yetkilendirme (authorization) süreçleri Flask'ın yerleşik oturum yönetimi (session management) mimarisine dayandırılarak tasarlanmıştır.

2.4.1. Şifreleme ve Kimlik Doğrulama (Authentication)

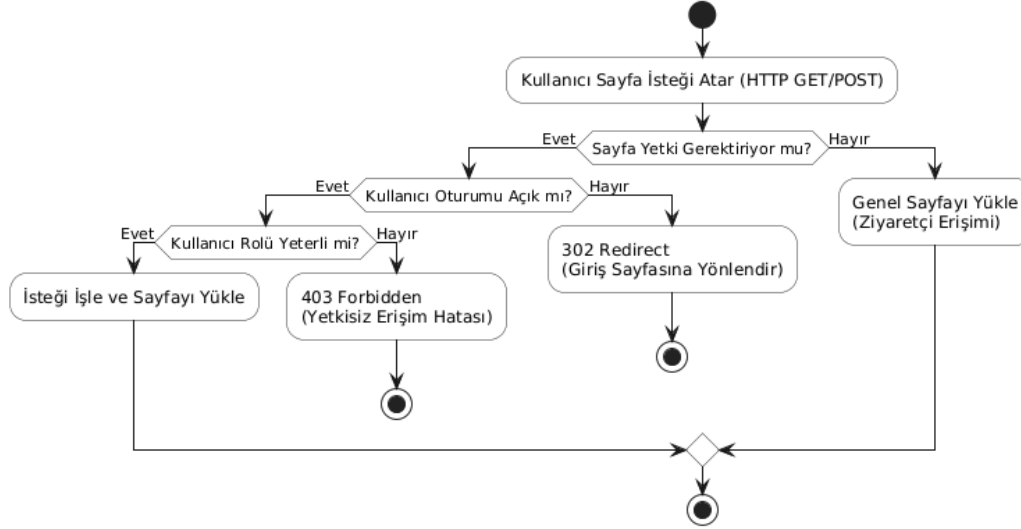
Kullanıcı şifreleri veritabanında kesinlikle düz metin (plain text) olarak saklanmaz. Kullanıcı kayıt olduğu anda şifresi, **Werkzeug.security** kütüphanesindeki `generate_password_hash` fonksiyonu ile kriptografik olarak özetlenir (hashlenir) ve "salt" (tuzlama) tekniği ile güçlendirilir. Giriş işlemlerinde ise `check_password_hash` metodu kullanılarak güvenli doğrulama sağlanır. Başarılı giriş sonrasında kullanıcıya güvenli bir oturum (Session) çerezi (cookie) atanır.

2.4.2. Rol Tabanlı Yetkilendirme (Role-Based Authorization)

Sistemde işlem yapabilme kapasitesi, üç farklı kullanıcı rolüne göre sınırlandırılmıştır:

1. **Ziyaretçiler (Guest):** Sisteme giriş yapmamış anonim kullanıcılarıdır. Sadece okuma yetkisine (Read-Only) sahiptirler. Popüler filmleri listeleyebilir ve diğer kullanıcıların yorumlarını okuyabilirler.

2. **Kayıtlı Kullanıcılar (Registered User):** Oturum açmış standart kullanıcılardır. Film arama, kendi kütüphanelerine film ekleme, puan verme ve yorum yapma (Create, Read, Update, Delete - CRUD işlemleri) yetkilerine sahiptirler. Yalnızca kendi oluşturdukları veriler üzerinde değişiklik yapabilirler.
3. **Yöneticiler (Admin):** Platformun genel düzenini sağlayan üst düzey kullanıcılardır. Tüm yorumları denetleme, uygunsuz içerikleri silme ve kullanıcı hesaplarını yönetme yetkisine sahiptirler.



2.4.3. Veritabanı ve Ağ Güvenliği

Sistem, web uygulamalarına yönelik yaygın saldırı vektörlerine karşı şu önlemleri barındırır:

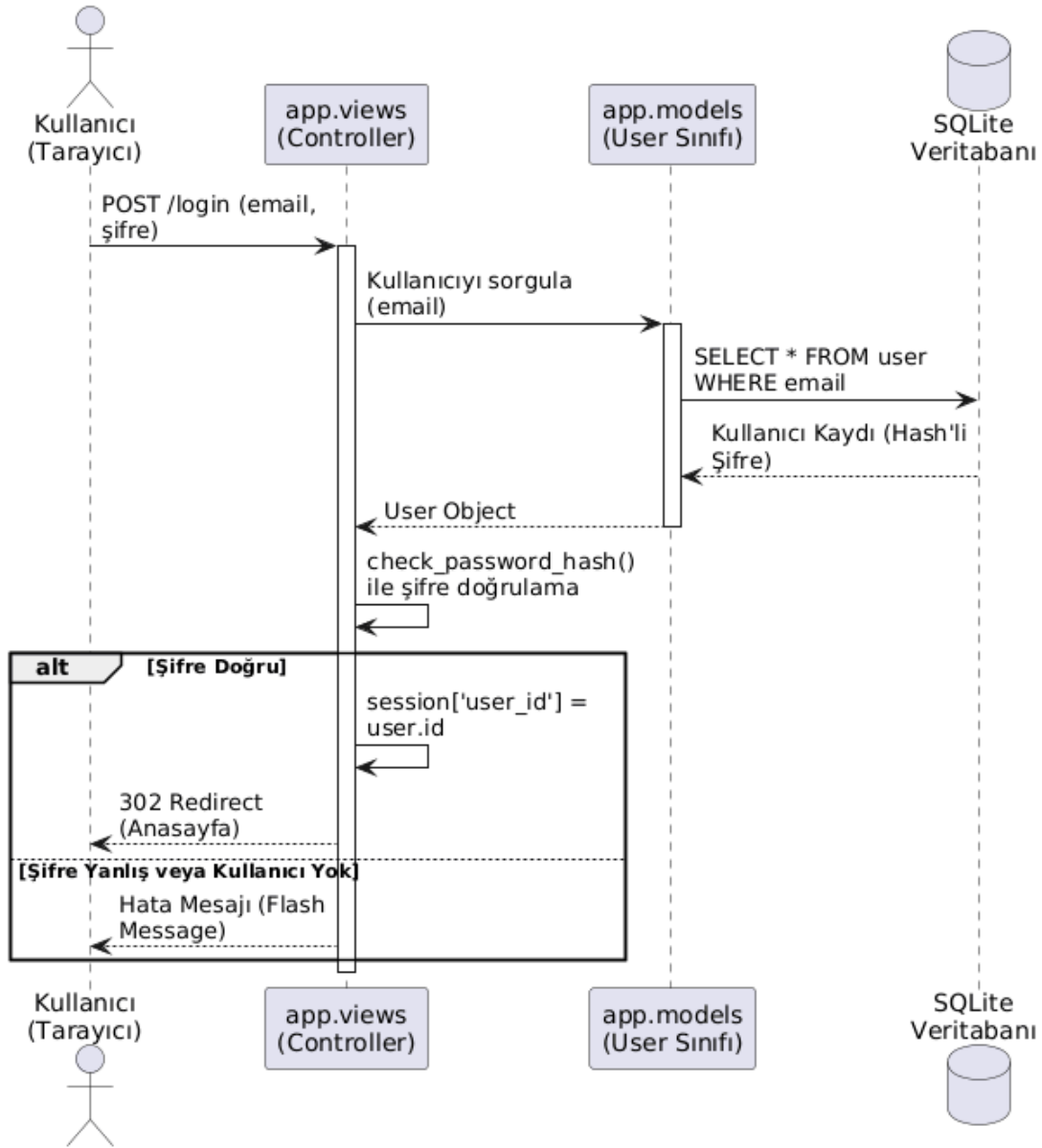
- **SQL Enjeksiyonu (SQL Injection) Koruması:** Doğrudan SQL sorguları yazmak yerine **SQLAlchemy ORM** kullanılması, dışarıdan gelen kullanıcı girdilerinin (input) veritabanına ulaşmadan önce otomatik olarak temizlenmesini (sanitize) sağlar.
- **CSRF (Cross-Site Request Forgery) Koruması:** Form gönderimlerinde Flask'ın sağladığı gizli anahtar (SECRET_KEY) altyapısı kullanılarak sahte isteklerin önüne geçilir.

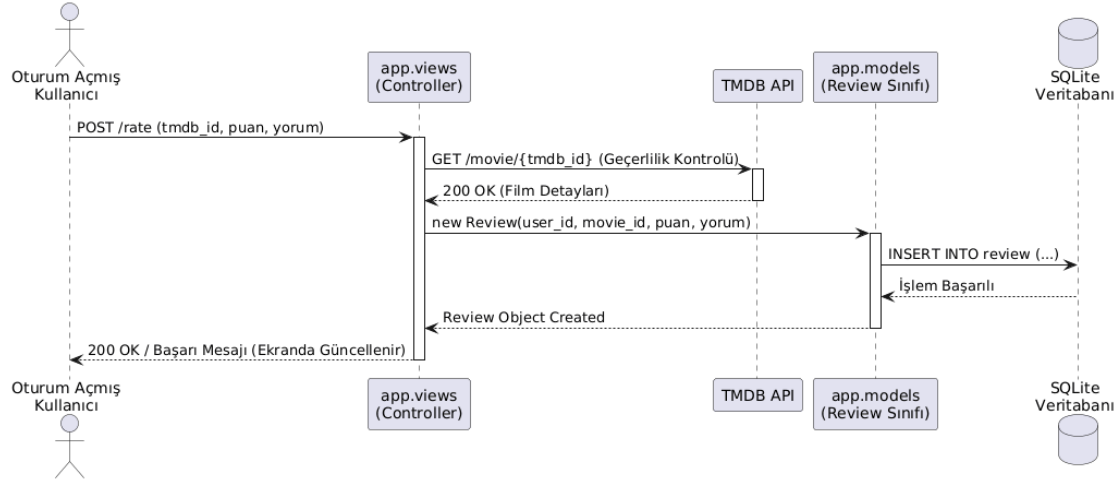
2.5. Küresel Kontrol Akışı (Global Control Flow)

CineRate sistemi, doğası gereği olay güdümlü (event-driven) ve **HTTP İstek-Cevap (Request-Response)** döngüsü üzerine kurulu bir mimariye sahiptir. Sistemin küresel kontrolü, istemciden (web tarayıcısı) gelen asenkron veya senkron olayların Flask uygulama sunucusu tarafından işlenmesiyle sağlanır.

Kontrol akışı merkezi bir denetleyici (Controller - views.py) üzerinden yönetilir. Gelen HTTP istekleri (GET, POST vb.), ilgili fonksiyonlara yönlendirilir (Routing). Denetleyici, isteğin türüne göre veritabanı işlemlerini (models.py) veya harici API çağrılarını (TMDB Client) koordine eder ve elde ettiği sonuçları sunum katmanına (Jinja2 Templates) aktararak kontrolü tekrar istemciye bırakır.

Bu kontrol akışını teknik düzeyde daha iyi ifade edebilmek için temel kullanım senaryolarına ait **Ardışık Düzen Diyagramları (Sequence Diagrams)** aşağıda sunulmuştur:





2.6. Sınır Koşulları (Boundary Conditions)

Sistemin normal çalışma rutini dışındaki başlangıç, bitiş ve istisnai durumlara karşı sergileyeceği davranış modelleri aşağıda tanımlanmıştır.

2.6.1. Başlatma (Initialization)

Sistem başlatıldığında create_app() uygulama fabrikası (application factory) tetiklenir. Bu süreçte sırasıyla şu işlemler gerçekleşir:

1. Flask konfigürasyonları (Secret Key, Database URI) belleğe yüklenir.
2. SQLAlchemy veritabanı motoru başlatılır ve db.create_all() komutu ile mevcut veritabanı tablolarının (User, Movie, Review) bütünlüğü kontrol edilir; eğer tablolar yoksa fiziksel SQLite dosyası (.db) içinde yeni tablolar oluşturulur.
3. Modüller (Blueprints) ana uygulamaya kaydedilir ve sunucu HTTP isteklerini dinlemeye (listening) hazır hale gelir.

2.6.2. Sonlandırma (Termination)

Uygulama sunucusunun kapatılması (shutdown) durumunda sistem "zarif sonlandırma" (graceful shutdown) prensibine göre hareket eder. SQLAlchemy, askıda kalan veritabanı işlemlerini (commit) tamamlar veya hatalı olanları geri alır (rollback). Kullanıcı oturum verileri çerezlerde (cookies) tutulduğu için sunucu yeniden başlatıldığında geçerliliğini korur (oturum süresi dolmadığı sürece).

2.6.3. Hata Durumları (Failure)

Sistem çalışırken karşılaşılabilecek olası kriz anları ve sistemin vereceği tepkiler şu şekildedir:

- **TMDB API Kesintisi:** Dış servis olan TMDB'nin yanıt vermemesi (timeout) durumunda sistem çökmez (crash). İstekler try-except blokları içinde yönetilir ve API'ye ulaşamadığında kullanıcıya "Film bilgileri şu an alınamıyor, lütfen daha sonra tekrar deneyiniz" şeklinde kontrollü bir geri bildirim (Flash Message) sunulur.
- **Veritabanı Hataları:** Veritabanına yazma sırasında oluşabilecek kilitlenmelerde (database lock), işlem iptal edilerek (db.session.rollback()) veri tutarsızlığının önüne geçilir.
- **Geçersiz Kullanıcı Girdileri:** Kullanıcıların formlara hatalı (Örn: metin yerine özel karakterler) veri girmesi durumunda Flask-WTF veya manuel backend kontrolleri ile istek reddedilir ve kullanıcıya anında uyarı verilir.

3. Alt Düzey Tasarım

Bu bölümde, CineRate sisteminin nesne yönelimli tasarımı, veritabanı şeması ve yazılım bileşenlerinin detaylı yapısı ele alınmaktadır.

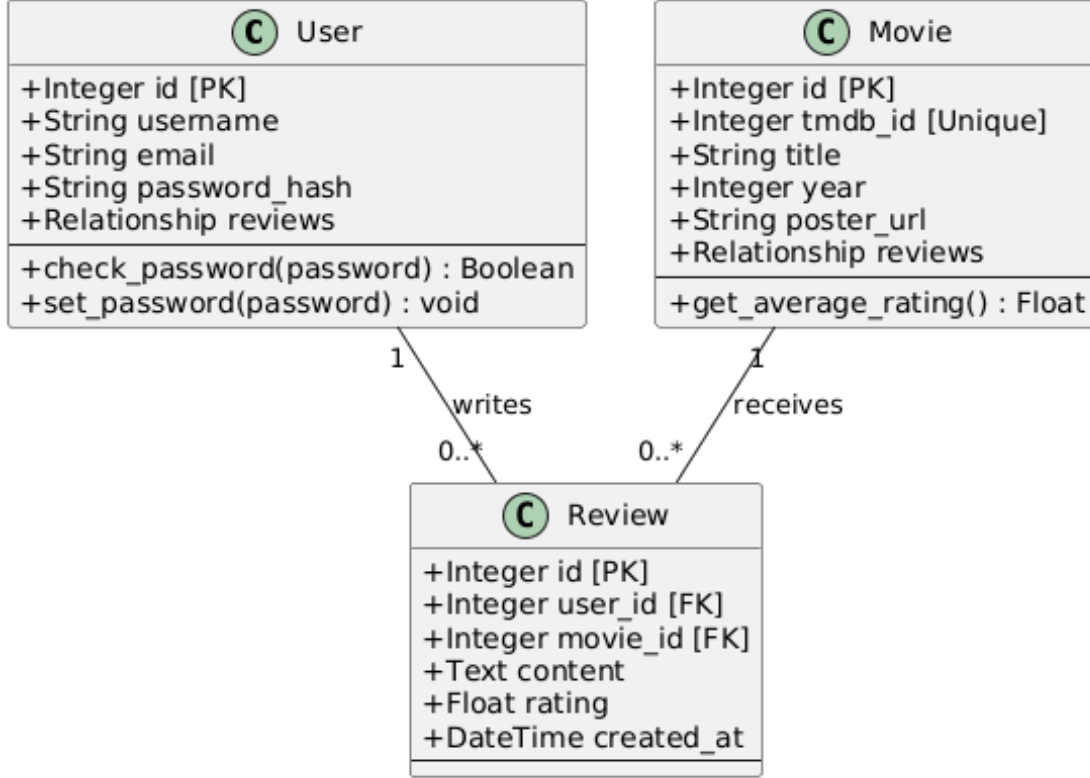
3.1. Nesne Tasarım Kararları (Design Trade-offs)

Sistemin mimarisi kurgulanırken belirli mühendislik tercihleri yapılmış ve bu tercihler sırasında çeşitli ödünleşimler (trade-offs) değerlendirilmiştir:

- **SQLite vs. PostgreSQL (Taşınabilirlik vs. Ölçeklenebilirlik):** Projede kurulum gerektirmeyen ve uygulama dosyasıyla birlikte taşınabilen SQLite tercih edilmiştir. Bu, geliştirme hızını ve taşınabilirliği (portability) artırırken; çok yüksek trafikli senaryolarda PostgreSQL'in sunduğu ileri düzey eşzamanlılık desteğinden feragat edilmiştir.
- **API Entegrasyonu vs. Yerel Veri Depolama (Güncellik vs. Performans):** Film verileri için yerel bir veritabanı oluşturmak yerine TMDB API kullanımı tercih edilmiştir. Bu karar, veritabanı bakım maliyetini düşürüp verilerin her zaman güncel kalmasını sağlarken; API'ye olan ağ bağımlılığını (latency) ve harici servis kesintisi riskini beraberinde getirmektedir.
- **ORM (SQLAlchemy) vs. Raw SQL (Geliştirme Hızı vs. Ham Performans):** Veritabanı işlemleri için SQLAlchemy ORM katmanı seçilmiştir. Bu sayede kodun okunabilirliği ve güvenliği (SQL Injection koruması) artırılmış, ancak çok karmaşık sorgularda ham SQL'in sunduğu mikrosaniye düzeyindeki performans avantajından ödün verilmiştir.

3.2. Son Nesne Tasarımı (Detailed Class Design)

Analiz raporundaki kavramsal sınıflar, tasarım aşamasında kod seviyesine (Design-Level) çekilmiştir. Aşağıdaki **Ayrıntılı UML Sınıf Diyagramı**, models.py dosyasındaki gerçek Python sınıflarını, veri tiplerini ve aralarındaki teknik ilişkileri göstermektedir.



Bu tasarımda, analiz aşamasından farklı olarak sınıfların içindeki her bir öznitelik (attribute) ve metot, Flask-SQLAlchemy yapısına uygun olarak tanımlanmıştır. Örneğin, User sınıfındaki password_hash alanı güvenli giriş yönetimini temsil ederken; Movie sınıfındaki tmdb_id alanı harici API ile senkronizasyonu sağlamaktadır.

3.3. Paketler / Katmanlar (Packages / Layers)

CineRate sistemi, sorumlulukların ayrılması (Separation of Concerns) prensibine uygun olarak katmanlı bir paket yapısında organize edilmiştir. Bu yapı, hem geliştirme sürecini kolaylaştırmakta hem de sistemin bakımını daha güvenli hale getirmektedir.

3.3.1. Dış Paketler (External Packages)

Sistemin altyapısını oluşturan ve dışarıdan entegre edilen kütüphanelerin rolleri aşağıda belirtilmiştir:

- **Flask Framework:** Uygulamanın ana iskeletini ve HTTP yönlendirme (routing) mekanizmasını sağlar.
- **SQLAlchemy (ORM):** Veritabanı tabloları ile Python nesneleri arasındaki eşleşmeyi yönetir.
- **Requests Kütüphanesi:** TMDB API ile asenkron veri alışverişini ve HTTP isteklerini yönetir.
- **Werkzeug:** Güvenli şifre özetleme (hashing) ve doğrulama işlemleri için kriptografik araçlar sunar.

3.3.2. İç Paketler (Internal Packages)

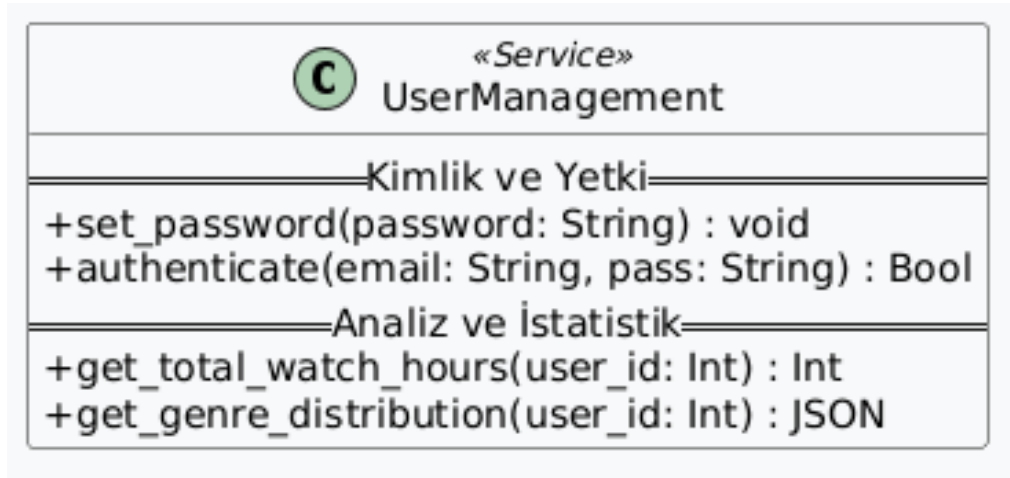
Kendi geliştirdiğimiz ve sistemin iş mantığını barındıran paketlerin sorumlulukları şunlardır:

- **models Paketi:** Veritabanı şemalarını ve veri varlıklarını (User, Movie, Review) barındırır.
- **views Paketi:** Kullanıcıdan gelen istekleri karşılayan, API çağrılarını koordine eden ve mantıksal süreçleri (puan verme, arama vb.) yürüten denetleyicileri içerir.
- **templates Paketi:** Kullanıcı arayüzünü oluşturan dinamik Jinja2 şablon dosyalarını barındırır.
- **static Paketi:** Sistemin görsel kimliğini oluşturan CSS dosyalarını, JavaScript scriptlerini ve yerel görselleri saklar.

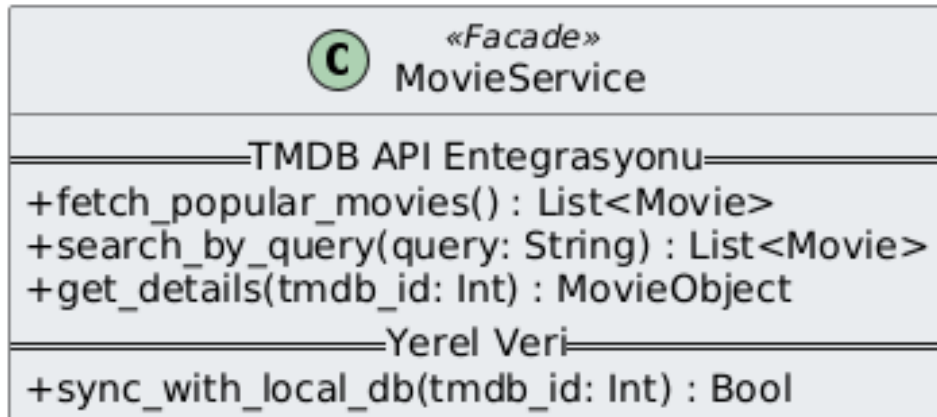
3.4. Sınıf Arayüzleri (Class Interfaces)

Bu bölüm, sistemdeki kritik sınıfların sahip olduğu metotların teknik detaylarını ve operasyonel görevlerini tablo halinde sunmaktadır.

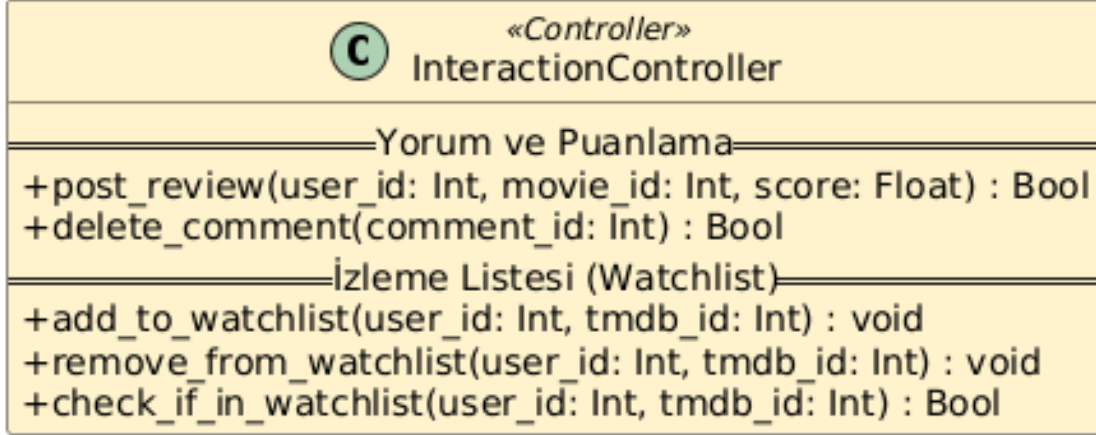
3.4.1. User (Kullanıcı) Sınıfı Metotları



3.4.2. Movie (Film) ve API Servis Metotları



3.4.3. Interaction (Etkileşim) Metotları



3.5. Tasarım Desenleri (Design Patterns)

CineRate projesinin geliştirme sürecinde, kodun okunabilirliğini artırmak, bileşenler arasındaki bağımlılığı optimize etmek ve yaygın yazılım sorunlarına standart çözümler üretmek amacıyla çeşitli tasarım desenleri uygulanmıştır.

3.5.1. Model-View-Template (MVT) Deseni

Flask framework'ünün temelini oluşturan bu desen, projenin en üst seviyedeki organizasyon prensibidir. Klasik MVC (Model-View-Controller) deseninin bir varyasyonu olan MVT, projemizde şu şekilde hayat bulmuştur:

- **Model:** models.py içindeki veritabanı sınıfları veriyi temsil eder.
- **View:** views.py içindeki rotalar, iş mantığını ve kontrolü (Controller görevi) üstlenir.
- **Template:** HTML dosyaları, kullanıcıya sunulacak görselliği yönetir. Bu desen sayesinde arayüz değişiklikleri, arka plandaki veri mantığını etkilemeden gerçekleştirilebilmektedir.

3.5.2. Facade (Cephe) Deseni

Sistemimizde TMDB API ile olan iletişim, Facade tasarımı prensibiyle kurgulanmıştır. API'nin sunduğu karmaşık HTTP istekleri, JSON ayrıştırma işlemleri ve hata kontrolleri; sistemin geri kalanı için sadeleştirilmiş bir arayüz (Örn: `get_movie_details`) arkasına gizlenmiştir.

- **Uygulama:** API servis modülü, karmaşık dış dünya işlemlerini sistemin diğer bileşenlerinden izole ederek "tek bir pencere" üzerinden veri sunar.

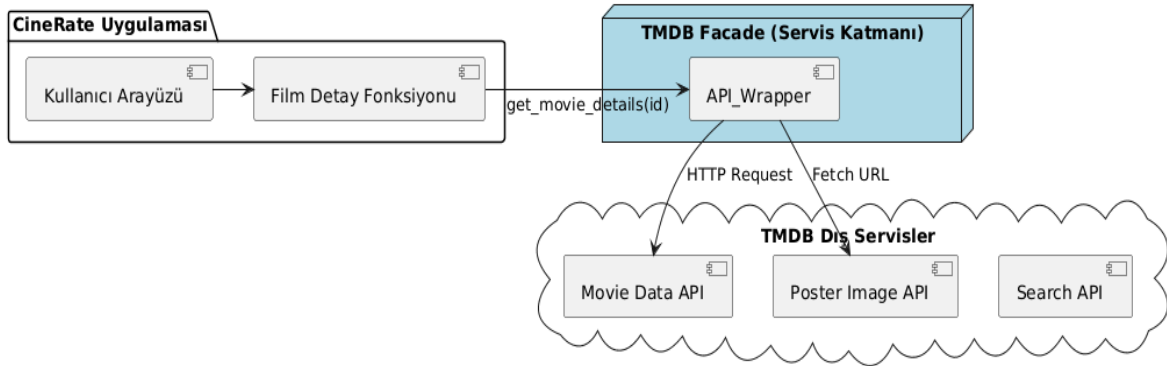
3.5.3. Singleton (Tek Nesne) Deseni

Veritabanı bağlantısının yönetimi için Singleton prensibi benimsenmiştir. Uygulama çalıştığı sürece veritabanı motorunun (SQLAlchemy Engine) bellekte sadece bir kez yaratılması ve tüm sistem tarafından bu tekil örnek (instance) üzerinden işlem yapılması sağlanmıştır.

- **Uygulama:** db nesnesi, uygulamanın her yerinde aynı bağlantı havuzunu kullanarak kaynak tüketimini minimize eder ve veri tutarlılığını korur.

3.5.4. Proxy (Vekil) Deseni

Özellikle film afişlerinin (`poster_url`) sunulması sürecinde bu desenin mantığından yararlanılmıştır. Sistem, poster görüntüsünü kendi bünyesinde barındırmak yerine, TMDB sunucularındaki görselin adresini bir vekil gibi kullanarak istemciye iletir. Böylece yerel depolama alanı verimli kullanılırken, veri akışı dinamik olarak yönetilir.



Sözlük (Glossary)

API (Application Programming Interface): Farklı yazılımların birbirleriyle iletişim kurmasını sağlayan arayüz. Projede, dışarıdan güncel film verilerini çekmek için TMDB API kullanılmıştır.

ORM (Object-Relational Mapping): Veritabanı tablolarını nesne yönelimli programlama (OOP) dillerindeki sınıflara dönüştüren teknik. Projede veritabanı işlemleri için **SQLAlchemy ORM** tercih edilmiştir.

MVT (Model-View-Template): MVC (Model-View-Controller) mimarisinin Flask framework'üne özgü uyarlamasıdır. Veri (Model), iş mantığı (View) ve sunum (Template) katmanlarının ayrıştırılmasını ifade eder.

JSON (JavaScript Object Notation): Sistemler arası veri transferinde kullanılan hafif ve okunabilir bir veri formatı. TMDB API'den gelen film detayları bu formatta işlenir.

Jinja2: Flask uygulamalarında kullanılan, HTML dosyaları içine Python kodları (döngüler, if blokları) yazılmasına olanak tanıyan dinamik şablon (template) motoru.

Hashing (Özetleme): Kullanıcı şifrelerinin güvenliğini sağlamak amacıyla geri döndürülemez (tek yönlü) bir şifreleme algoritmasından geçirilmesi işlemidir (Werkzeug kütüphanesi ile sağlanır).

Session (Oturum): Kullanıcıların sisteme giriş yaptıktan sonraki durumlarını tarayıcı tarafında güvenli bir şekilde takip etmeye yarayan mekanizma.

Referanslar

- 1-) The Movie Database (TMDB), "TMDB API Developers Documentation," [Çevrimiçi]. Erişim: <https://developers.themoviedb.org/>
- 2-) Pallets Projects, "Flask Web Development Framework Documentation," [Çevrimiçi]. Erişim: <https://flask.palletsprojects.com/>
- 3-) SQLAlchemy, "The Python SQL Toolkit and Object Relational Mapper," [Çevrimiçi]. Erişim: <https://docs.sqlalchemy.org/>
- 4-) G. Booch, J. Rumbaugh, ve I. Jacobson, *The Unified Modeling Language User Guide*, 2. Baskı, Addison-Wesley, 2005. (UML tasarım standartları referansı)